American University of Beirut
Maroun Semaan Faculty of Engineering and Architecture
Department of Electrical and Computer Engineering

EECE 423/623: Reconfigurable Computing
Project #1: AXI-Lite Accelerometer IP Block

Introduction

The aim of this project is to develop an AXI-Lite SPI master block, its driver functions, and a test application. The SPI master block will be used to interface with a Digilent PmodACL 3-axis accelerometer through one of the Pmod connectors on the the ZedBoard.

The Serial Peripheral Interface (SPI)

SPI is a synchronous serial communication protocol commonly used to connect digital devices across short distances. SPI devices communicate in full duplex mode using a master-slave architecture. Figure 1 shows the SPI interface, which consists of four wires:

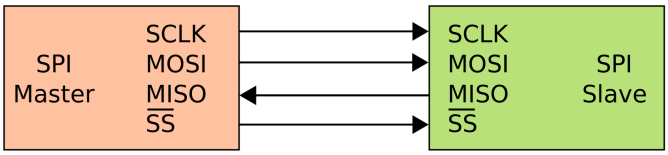

Figure 1: SPI Master-Slave Architecture (Wikipedia)

- `SCLK` is the serial clock. It synchronizes the transfer of one bit between the master and slave devices. `SCLK` is only active during a data transfer, and is idle otherwise.
- `MOSI` is the master-out-slave-in signal. It is used to transfer serial data from the master to the slave.
- `MISO` is the master-in-slave-out signal. It is used to transfer serial data from the slave to the master.
- $\overline{\texttt{SS}}$ is the active-low slave-select signal. It is asserted for the duration of a data transfer, and is idle otherwise. Slave select is often referred to as chip select ($\overline{\texttt{CS}}$).

Data transfers are initiated by the master device. A transfer begins when the chip-select signal is asserted. Data is then transferred serially (one bit at a time) over the `MOSI` and `MISO`

wires. Because it takes one SCLK cycle to transfer a bit, transferring n bits takes n clock cycles. Data is typically transferred one byte at a time, so a data transfer typically consumes eight clock cycles.

The MOSI and MISO wires are connected through shift registers in the master and slave devices. Each shift register can be initialized by the corresponding master or slave device by loading its flip-flops with data in parallel. When a bit is transferred from the master to the slave over the MOSI wire, it is shifted out of the master shift register and into the slave shift register. This in turn causes a bit to be shifted out of the slave shift register and into the master shift register over the MISO wire. So when a master sends a byte to a slave, it also receives a byte from the slave, which it ignores. Similarly, when a master reads a byte from a slave, it also sends a byte to the slave, which the slave ignores. Figure 2 shows the coupling between a master and a slave through the MOSI and MISO wires, and their respective shift registers.

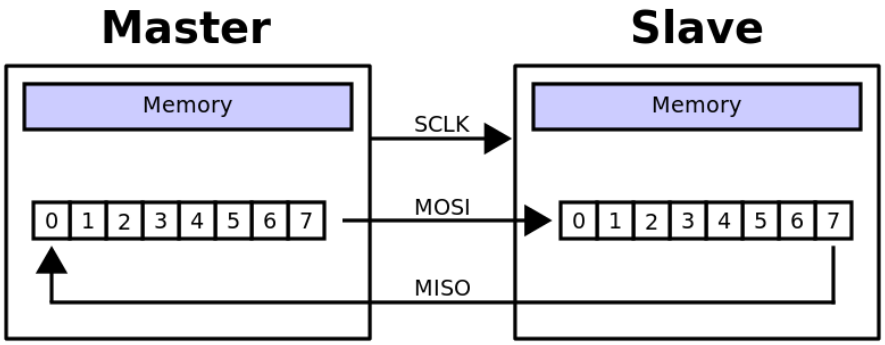

Figure 2: SPI Master-Slave Coupling

SPI defines four data transfer modes based on clock polarity (CPOL) and phase (CPHA). These define the clock idle state, and the clock edge where data is sampled. The choice of data transfer mode is device specific and depends on the device manufacturer. Most SPI slave devices use a fixed data transfer mode that is specified in the device data sheet, while most SPI master devices can be programmed to support different data transfer modes. Figures 3 and 4 show the different SPI data transfer modes.

CPOL	CPHA	Clock Idle	Data Sampled
0	0	Low	Rising edge
0	1	Low	Falling edge
1	0	High	Falling edge
1	1	High	Rising edge

Figure 3: SPI CPOL/CPHA Combinations

SPI Master

In this project, we will package a SPI master as an AXI-Lite IP block, and we will use it to interface with a 3-axis accelerometer. Figure 5 is a block diagram of the SPI master, which includes four signal groups: Control, transmitter (TX) interface, receiver (RX) interface, and SPI interface.

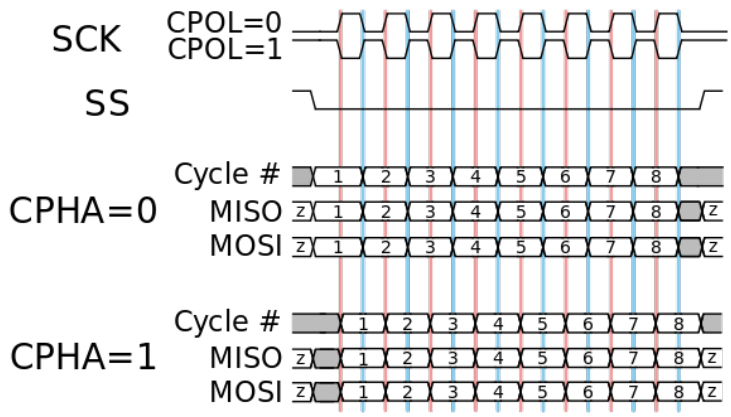

Figure 4: SPI Data Transfer Modes (Wikipedia)

Control Signals

- `i_Rst_L`: Active-low reset signal.
- `i_Clk`: SPI master clock. In our design we will be implementing the SPI master in the PL part of a Zynq device, so the SPI master clock will operate at 100 MHz. The SPI interface `SCLK` signal will be derived from this clock.
- `i_spi_mode`: A 2-bit vector (`CPOL` and `CPHA`) that specifies the SPI data transfer mode.
- `i_clk_scale`: A 16-bit vector for scaling the SPI master clock and deriving the `SCLK` signal. An `i_clk_scale` value of n divides the SPI master by the same factor. Note that the accelerometer we will be using should be operated at an `SCLK` frequency of 4 MHz.
- `i_cs_inactive_clks`: A 3-bit vector for specifying the minimum number of SPI master clock cycles between successive data transfers. This should be set to at least 1, and in our design we will also set it to 1.

TX Interface

- `i_TX_Count`: A 5-bit vector for specifying the number of multi-byte transfers per transaction. This value should be set at the start of a data transfer transaction, and should be a value between 1 and 16.
- `i_TX_Byte`: The current data byte to be sent to a slave device.
- `i_TX_DV`: The data-valid signal. Asserted for one SPI master clock cycle to indicate that the data on the `i_TX_Byte` port is valid. In a multi-byte transfer, this signal should be pulsed for every byte sent.
- `o_TX_Ready`: Indicates if the SPI master is ready to receive new data for transfer to the slave. During a data transfer it is set to 0 to indicate the SPI master is not ready. Otherwise it is set to 1.

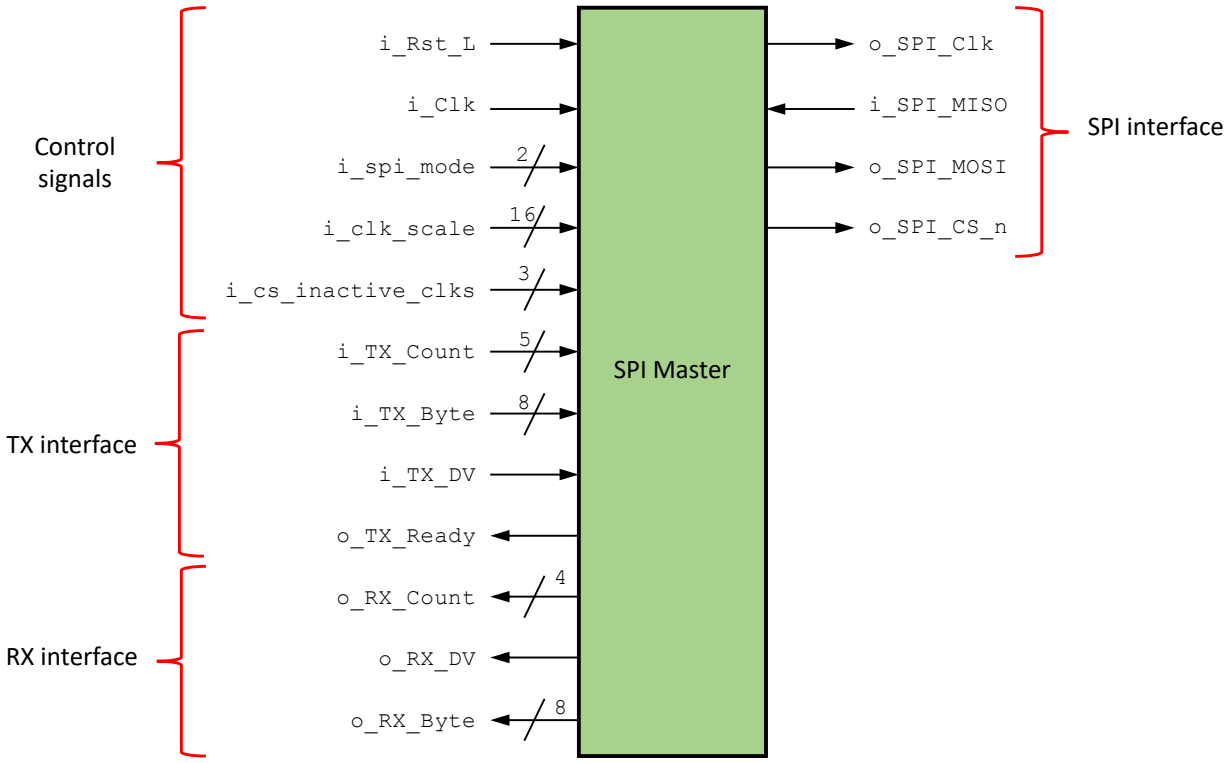

Figure 5: SPI Master

RX Interface

- `o_RX_Count`: A 4-bit vector for specifying the index of the data byte received in the current transaction. Should be a value between 0 and 15.
- `o_RX_DV`: The data-valid signal. Asserted for one SPI master clock cycle to indicate that the data on the `o_RX_Byte` port is valid. In a multi-byte transfer, this signal should be pulsed for each byte received in the current transaction.
- `o_RX_Byte`: The current byte received from the slave device.

SPI Interface

- `o_SPI_Clk`: The `SCLK` signal derived from the `i_Clk` signal and scaled by the value specified by the `i_clk_scale` signal.
- `i_SPI_MISO`: The `MISO` signal.
- `o_SPI_MOSI`: The `MOSI` signal.
- `o_SPI_CS_n`: The active-low $\overline{\texttt{CS}}$ signal.

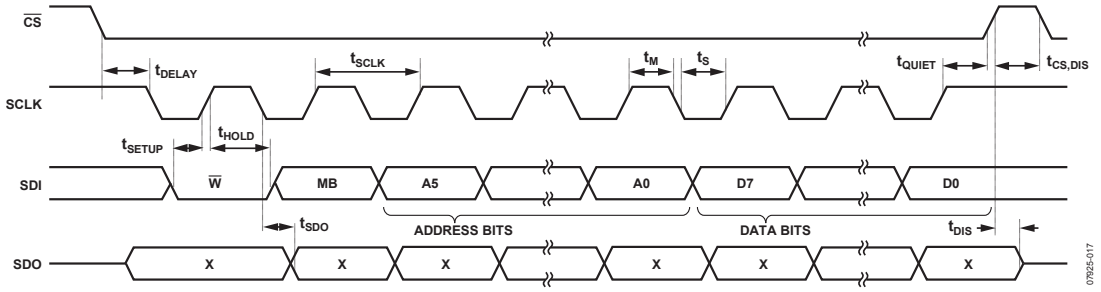

Figure 37. SPI 4-Wire Write

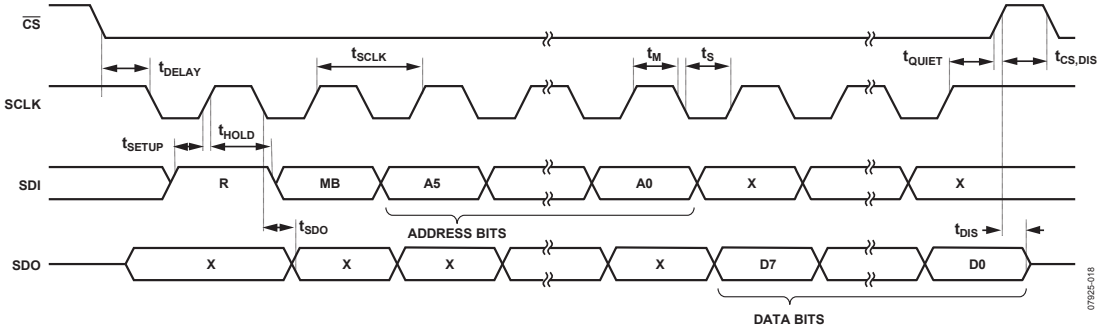

Figure 38. SPI 4-Wire Read

Figure 6: SPI Write and Read Data Transactions

Digilent PmodACL 3-Axis Accelerometer

We will connect the SPI master to a Digilent PmodACL 3-axis accelerometer through one of the Pmod connectors on the ZedBoard. The PmodACL uses an Analog Devices ADXL345 digital accelerometer that will act as a SPI slave, and that we will use to measure acceleration along the X, Y, and Z axes. Acceleration is expressed in units of gravitational acceleration g. The ADXL345 can be configured to measure accelerations in the range of ± 2 g to ± 16 g, and it can return measured values with a resolution of 10-13 bits per sample. Please refer to the ADXL345 data sheet for further details. You can find more information about the Digilent PmodACL including a link to the ADXL345 data sheet at:

`https://reference.digilentinc.com/reference/pmod/pmodacl/reference-manual`

The ADXL345 uses SPI data transfer mode 3 (`CPOL=1` and `CPHA=1`) to communicate with a SPI master. It also contains 29 8-bit configuration and data registers that are accessed through two-byte transactions. To initiate a data transfer, the master sends an 8-bit command byte containing the type of operation to perform (read vs. write), whether the transfer involves multiple bytes, and a 6-bit register address. For write operations, the master sends a second 8-bit byte containing the data to be stored in the register. For read operations, the master sends a dummy byte so that the slave can send the required data to the master. Figure 6 shows the timing diagrams for single data-byte write and read transactions.

To read or write multiple bytes in the same transaction, the multiple-byte bit, located after the R/$\overline{\text{W}}$ bit in the first byte transfer, must be set. After the register addressing and the first byte of data, each subsequent set of clock pulses (eight clock pulses) causes the ADXL345 to point to

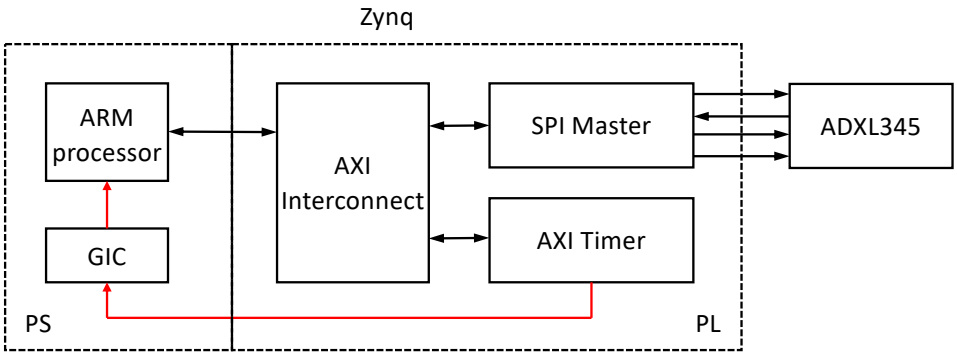

Figure 7: System Hardware Design

the next register for a read or write automatically. This shifting continues until the clock pulses end and $\overline{CS}$ is deasserted. To perform reads or writes on different, nonsequential registers, $\overline{CS}$ must be deasserted between transmissions and the new register must be addressed separately.

Before we can start using the ADXL345, we must first initialize it. This involves setting different control registers immediately after the device is powered up. A minimal initialization sequence involves setting control registers `DATA_FORMAT` (address `0x31`) to `0x09`, and `POWER_CTL` (address `0x2D`) to `0x08`, respectively. These configure the accelerometer to start measuring accelerations in the range ± 4 g along the three axes with a precision of 11 bits. More details about ADXL345 control registers can be obtained from its data sheet.

The actual measurements can be read from registers `DATAX0`, `DATAX1`, `DATAY0`, `DATAY1`, `DATAZ0`, and `DATAZ1` (addresses `0x32` to `0x37`). Each pair of registers returns the acceleration along a given axis. Acceleration values are stored as 11-bit signed integers that are sign-extended to 16 bits. Registers `DATAX0`, `DATAY0`, and `DATAZ0` store the low-order 8 bits of an acceleration measurement, while registers `DATAX1`, `DATAY1`, and `DATAZ1` store the upper 8-bits.

System Hardware

Figure 7 is a block diagram of the system hardware design. In addition to the AXI-Lite SPI master IP block, the system includes an AXI Timer that should be programmed to generate periodic interrupts at 500 msec intervals. Every time an interrupt is generated, the interrupt handler should read new measurements from the ADXL345 and store them in appropriate global variables that can be used in a main program. The AXI-Lite SPI master should be physically attached to the Digilent PmodACL module using Pmod connector JA1, which requires paying attention to the following:

- The PmodACL module should be attached to Pmod connector JA1 with the module jumpers and integrated circuit facing upwards.
- The SPI signals should be added as ports to the top-level entity of the AXI-Lite SPI master block. Unlike previous lab experiments where top-level entities were only attached to the AXI interconnect, the top-level entity for the AXI-Lite SPI master block will also be attached to an external peripheral device. This requires marking these ports **external** in the Vivado block diagram. This can be achieved by right-clicking on each port and selecting **Make external**.

- You will also have to create a constraints file to assign the external SPI ports to the Zynq device pins connected to Pmod connector JA1. Please refer to the ZedBoard Hardware User's Guide to identify the appropriate Zynq device pins.

System Software

Xilinx Vivado will generate a basic device driver that will enable you to read and write to the AXI-Lite interface registers of the SPI master device. Before you start implementing your design, decide how many registers you will need, and how the registers will be used to both configure and transfer data to and from *any* SPI slave device. You must also develop driver functions that will enable you to transfer an arbitrary number of bytes between the master and slave devices. You can test your data read function by reading the ADXL345 device ID, which is stored in the `DEVID` register (address `0x00`) and has a value of `0xE5`.

In addition to the SPI master driver functions, you must develop an application program that implements a digital level. A digital level is a tool that returns the angle of an incline in units of degrees. Figure 8 shows the gravitational forces along each of the three axes when the ADXL345 is in different positions in space. Please use this information to calculate the inclination of the ZedBoard in the Y-Z plane. You can display the inclination value on the PuTTY terminal screen. To identify the X, Y, and Z axes for the Pmod module locate the white dot on top of the ADXL345 integrated circuit, and compare it to Figure 8. Finally, your application program must also initialize the SPI master, the SPI slave, and the AXI Timer. It must also initialize AXI Timer interrupts and include an appropriate AXI Timer interrupt handler.

Deliverables

Please upload the following source files to Moodle by 11:59 pm on Friday, November 7, 2025:

1. Your `*_v1_0.v` top-level module file.
2. Your `*_v1_0_S00_AXI.v` file. Please include a comments section that describes how you used your interface registers.
3. Your `*.h` and `*.c` SPI master driver sources files.
4. Your `*.c` application source file.
5. A project report describing your design. Your report must include a block diagram

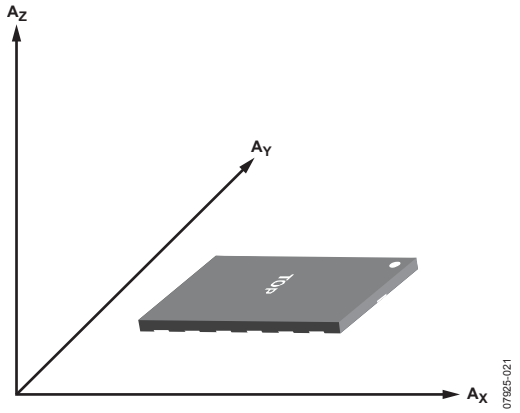

Figure 57. Axes of Acceleration Sensitivity (Corresponding Output Voltage Increases When Accelerated Along the Sensitive Axis)

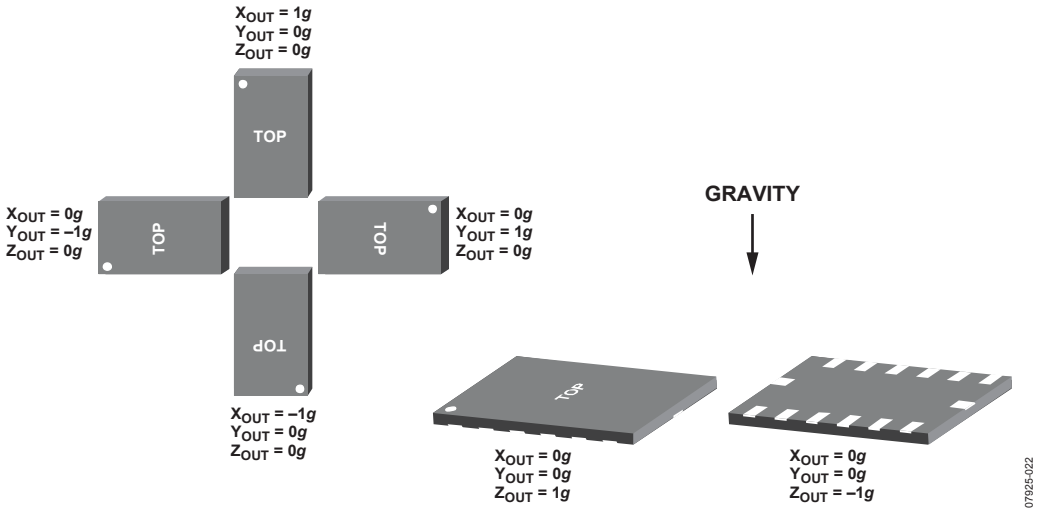

Figure 58. Output Response vs. Orientation to Gravity

Figure 8: Gravitational forces along the X, Y, and Z axes.